

Render Deployment Guide

Kansas Data Science Education Atlas

Last Updated: April 2026

Preface

This guide documents the complete process for deploying the Kansas Data Science Education Atlas to Render, including the FastAPI backend and the React/Vite frontend. This serves as a backup deployment independent of the KSU server infrastructure.

The deployment consists of three Render components:

- A PostgreSQL database hosted on Render
- A FastAPI backend web service hosted on Render
- A React/Vite frontend hosted as a Render Static Site

This guide assumes that you have a delineation between your frontend and backend code. In this project they are separated as frontend/ and backend/ subdirectories inside the project root folder. This guide also assumes you have a Mac. For any step that does not match your specifications, please do the equivalent according to your project,

Note: Render free tier instances spin down after periods of inactivity. The first request after inactivity may take 30-60 seconds. This is expected behavior on the free tier.

How to Read This Guide — Terminal Locations

Every command in this guide is labeled with a colored bar showing exactly where to run it. Before reading further, familiarize yourself with these locations:

Terminal Location Reference

Throughout this guide, every command block is labeled with where to run it. Here is what each label means:

► **VS Code Terminal (project root)** Open your project in VS Code. In the top menu, go to Terminal > New Terminal. The terminal opens at the root of your project folder (the folder that contains frontend/ and backend/ subdirectories). This is the most common location.

► **VS Code Terminal (backend/)** Same as above, but navigate into the backend subfolder first: `cd backend`

► **VS Code Terminal (frontend/)** Same as above, but navigate into the frontend subfolder first: `cd frontend`

► **Mac Terminal (any location)** Open the Mac Terminal app (Spotlight > Terminal). Location does not matter for these commands, typically curl or psql commands hitting remote URLs.

Tip: In VS Code, you can open multiple terminal tabs by clicking the + button in the terminal panel. This is useful when you need one terminal running the backend server and another for other commands.

Prerequisites

Before starting, ensure you have the following ready:

- A Render account at render.com — sign up with your GitHub account for easiest integration
- Your project repository pushed to GitHub
- Your project open in VS Code
- Original CSV data files (dataset1.csv through dataset7.csv) present in data/raw/ inside your project
- PostgreSQL installed locally — verify by running:

► **Run in: Mac Terminal (any location)**

```
psql --version
```

- Node.js and npm installed locally — verify by running:

► **Run in: Mac Terminal (any location)**

```
node --version  
npm --version
```

Your project folder structure should look like this:

► **Run in: VS Code Terminal (project root)**

```
ls  
# Expected output includes: frontend/ backend/ data/
```

Step 1: Create a PostgreSQL Database on Render

All of Step 1 is done in the Render web dashboard at render.com. No terminal commands are needed.

1.1 Create the Database

1. Log in to render.com
2. Click New + in the top navigation bar
3. Select PostgreSQL from the dropdown menu

4. Fill in the database settings:
 - Name: atlas-db
 - Region: Oregon (US West) — or your nearest region
 - Plan: Free
5. Click Create Database
6. Wait approximately 1 minute for provisioning to complete

1.2 Retrieve Connection URLs

Once the database is ready, click on it in the Render dashboard. You will see two important connection URLs — copy both and save them somewhere accessible:

- External Database URL — used to load data from your local machine. Starts with `postgresql://...oregon-postgres.render.com/...`
- Internal Database URL — used by the Render backend service. Starts with `postgresql://...dpg-...` (shorter hostname)

Note: Always use the Internal Database URL for the backend service environment variable. Use the External URL only when running commands from your local machine.

Step 2: Load Data into the Render Database

All commands in Step 2 are run from your local machine inside VS Code. The project's virtual environment must be active.

Important: Before running these commands, verify that your `load_data.py` script reads `DATABASE_URL` from the environment rather than using hardcoded local credentials. Open `backend/scripts/load_data.py` in VS Code and confirm the connection is set up like this:

```
database_url = os.environ.get('DATABASE_URL',
                              'postgresql://localhost/atlas')
conn = psycopg2.connect(database_url)
```

If your script still has hardcoded `DB_PARAMS` with `localhost`, update it to use the environment variable above before proceeding.

2.1 Activate the Virtual Environment

All Python commands require the virtual environment to be active. From the project root:

```
► Run in: VS Code Terminal (project root)
source backend/venv/bin/activate
# You should see (venv) appear at the start of your terminal prompt
```

2.2 Create the Database Tables

Run the SQL schema file against the Render database. Replace `YOUR_EXTERNAL_DATABASE_URL` with the External Database URL you copied in Step 1.2:

► Run in: VS Code Terminal (project root)

```
psql YOUR_EXTERNAL_DATABASE_URL -f data/schemas/create_tables.sql
```

You should see repeated output like `DROP TABLE`, `CREATE TABLE`, `CREATE INDEX` — one set per table. No errors should appear.

2.3 Load the CSV Data

Pass the External Database URL as an environment variable directly to the load script. This avoids modifying your `.env` file:

► Run in: VS Code Terminal (project root)

```
DATABASE_URL=YOUR_EXTERNAL_DATABASE_URL python  
backend/scripts/load_data.py
```

The script will print progress as it inserts rows. When finished, you should see a summary table like this:

Key	Value
<code>institutions</code>	1425
<code>county_aggregations</code>	505
<code>courses</code>	175
<code>college_locations</code>	77
<code>digital_infrastructure</code>	105
<code>socioeconomic</code>	105
<code>atlas</code>	105

2.4 Verify the Data Loaded Correctly

Run a quick count to confirm the data is in the Render database:

► Run in: VS Code Terminal (project root)

```
psql YOUR_EXTERNAL_DATABASE_URL -c "SELECT count(*) FROM atlas;"
```

Expected result: `count = 105`. If the count is 0, the data did not load — re-run Step 2.3 and check for errors.

Step 3: Deploy the Backend Web Service

All of Step 3.1 through 3.2 is done in the Render web dashboard. Steps 3.3 and 3.4 involve your local code and terminal.

3.1 Create the Web Service on Render

7. In the Render dashboard, click New + and select Web Service
8. Connect your GitHub repository when prompted
9. Fill in the service settings:

Key	Value
Name	your-project-backend (this becomes part of the URL)
Root Directory	backend
Runtime	Python 3
Branch	main
Build Command	pip install -r requirements.txt
Start Command	uvicorn app.main:app --host 0.0.0.0 --port \$PORT
Instance Type	Free

3.2 Set Environment Variables in Render Dashboard

Scroll down to the Environment Variables section on the same page and add the following key-value pairs. Do not add quotes around the values:

Key	Value
DATABASE_URL	YOUR_INTERNAL_DATABASE_URL (from Step 1.2)
SECRET_KEY	your-secret-key-value
BACKEND_URL	https://your-service-name.onrender.com
BACKEND_CORS_ORIGINS	["https://your-frontend.onrender.com", "http://localhost:5173"]
EMAIL_CLIENT_ID	Your OAuth client ID from Google Cloud Console (ends in .apps.googleusercontent.com)
EMAIL_CLIENT_SECRET	Your OAuth client secret from Google Cloud Console (starts with GOCSPX-)
EMAIL_REFRESH_TOKEN	Your OAuth refresh token generated by running get_token.py (see Step 3.5 below)
FROM_EMAIL	your-email@gmail.com (the Gmail account used during OAuth authorization)

Note: Use the Internal Database URL for DATABASE_URL here — not the External URL. Internal connections are faster and more reliable between Render services.

Note: This project uses Gmail OAuth2 (not SMTP) to send emails. The GMAIL_CLIENT_ID, GMAIL_CLIENT_SECRET, and GMAIL_REFRESH_TOKEN values come from your Google Cloud project. See Step 3.5 below for full setup instructions. This is only required if your application includes email functionality such as the Data Request tab.

10. Click Deploy Web Service at the bottom of the page
11. Wait 3-5 minutes for the initial build to complete. You can watch the build log in the Render dashboard.

3.3 Verify the Backend is Running

Once the deployment shows as Live in the Render dashboard, verify it from your local terminal:

```
► Run in: Mac Terminal (any location)
curl https://your-service-name.onrender.com/health
```

Expected response: {"status": "healthy"}

Then verify the database connection is working:

```
► Run in: Mac Terminal (any location)
curl https://your-service-name.onrender.com/api/v1/counties | head -c 300
```

Expected response: JSON starting with {"total":105,"counties":[{"id":1,"county_name":"Allen County"...

3.4 Hardcode CORS in main.py

Open backend/app/main.py in VS Code. Find the CORSMiddleware block and hardcode the allowed origins directly. Using environment variables for CORS can cause silent parsing failures. Use this exact pattern:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "https://your-frontend.onrender.com", # add after Step 4
        "http://localhost:5173",
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Save the file. You will add the real frontend URL after completing Step 4. After saving, push the change to GitHub:

```
► Run in: VS Code Terminal (project root)
git add .
git commit -m "Hardcode CORS for Render deployment"
git push origin main
```

Render will auto-redeploy the backend within 2-3 minutes after the push.

3.5 Set Up Gmail OAuth for Email (Data Request Tab)

The Data Request tab sends confirmation emails with a ZIP download link to users. This uses Gmail OAuth2 — not SMTP — so no App Password is needed and emails can be sent to any recipient. Complete the following sub-steps once to configure it.

3.5.1 Create a Google Cloud Project and enable the Gmail API

1. Go to console.cloud.google.com and sign in with the Gmail account you want to send emails from.
2. Click the project dropdown at the top → New Project → name it (e.g. Kansas Atlas) → Create.
3. Go to APIs & Services → Library → search for Gmail API → click Enable.

3.5.2 Configure the OAuth Consent Screen

1. Go to APIs & Services → OAuth consent screen → select External → Create.
2. Fill in App name, User support email, and Developer contact email (all can be your Gmail). Click Save and Continue through the remaining screens.
3. Once the app is created, click Publish App to move it to production status. This ensures the refresh token never expires due to testing mode restrictions.

3.5.3 Create OAuth Credentials

1. Go to APIs & Services → Credentials → Create Credentials → OAuth client ID.
2. Select Desktop app as the application type → name it → Create.
3. Download the JSON file. Note the Client ID and Client Secret shown on screen — copy these to Render's Environment tab as `EMAIL_CLIENT_ID` and `EMAIL_CLIENT_SECRET`.

3.5.4 Generate the Refresh Token

Install the required library and run the token generation script from the project root:

```
► Run in: VS Code Terminal (project root)
pip install google-auth-oauthlib
python get_token.py
```

A browser window will open. Sign in with your Gmail account and click Allow. If you see a "Google hasn't verified this app" warning, click Advanced → Go to [app name] (unsafe) — this is expected since the app is your own project.

The terminal will print the refresh token. Copy it and add it to Render's Environment tab as `EMAIL_REFRESH_TOKEN`. Click Save Changes and wait for Render to redeploy.

Note: The refresh token lasts indefinitely as long as the OAuth app is in production mode and the token is used at least once every 6 months. Do not share or commit this token to GitHub. If you ever need to regenerate it, revoke the app at myaccount.google.com/permissions and re-run `get_token.py`.

3.6 Commit Dataset Files to GitHub

The Data Request tab packages and delivers the raw CSV files when a user submits a request. These files must be committed to GitHub so Render can access them. The CSV files live in `backend/data_raw/` and the code looks for them there via a path relative to `backend/app/api/data_request.py`.

Verify the files exist locally, then commit them:

```
▶ Run in: VS Code Terminal (project root)
ls backend/data_raw/
git add backend/data_raw/
git commit -m "Add dataset CSV files for data request download"
git push origin main
```

Note: The `data_raw/` folder path is resolved relative to `backend/app/api/data_request.py` using `parents[2]`. If you reorganize the project structure, update the `DATA_DIR` path accordingly in that file.

Step 4: Deploy the Frontend Static Site

Steps 4.1 and 4.2 involve editing local files in VS Code. Steps 4.3 and 4.4 are done in the Render dashboard.

4.1 Update Vite Base URL (VS Code)

Open `frontend/vite.config.ts` in VS Code. If you previously set `base: '/atlas/'` for the KSU deployment, change it to:

```
export default defineConfig({
  plugins: [react()],
  base: '/', // Use '/' for Render, '/PROJECT_NAME/' for KSU
  // ... rest of config
})
```

Note: Keep a comment noting that base must be changed back to `'/atlas/'` when deploying to the KSU server.

4.2 Update `.env.production` (VS Code)

Open `frontend/.env.production` in VS Code. Update the API URL to point to your Render backend:

```
VITE_API_URL=https://your-backend-service.onrender.com
```

Save the file, then rebuild the frontend locally to confirm it works:

► Run in: VS Code Terminal (frontend/)

```
cd frontend
npm run build
```

After the build completes, verify the correct backend URL was baked into the built files:

► Run in: VS Code Terminal (frontend/)

```
grep -r "your-backend-service" dist/
# Should output matching lines - confirms the URL is embedded correctly
```

4.3 Push Changes to GitHub

Go back to the project root and push everything:

► Run in: VS Code Terminal (project root)

```
cd ..
git add .
git commit -m "Configure frontend for Render deployment"
git push origin main
```

4.4 Create the Static Site on Render

12. In the Render dashboard, click New + and select Static Site
13. Connect your GitHub repository
14. Fill in these settings:

Key	Value
Name	atlas-frontend (or your preferred name)
Root Directory	frontend
Build Command	npm run build
Publish Directory	dist

15. Add an environment variable in the Environment section:

Key	Value
VITE_API_URL	https://your-backend-service.onrender.com

Important: Always set VITE_API_URL in Render's Environment panel, even if .env.production is already configured. Render injects it at build time on its servers, ensuring the correct URL is embedded in the deployed files.

16. Click Create Static Site

17. Wait 2-3 minutes for the build. Render will give you a public URL like `https://atlas-frontend-xxxx.onrender.com`

4.5 Update Backend CORS with the Frontend URL

Now that you have the real frontend URL, open `backend/app/main.py` in VS Code and add it to the CORS origins list:

```
allow_origins=[
    "https://atlas-frontend-xxxx.onrender.com", # your actual Render
    URL
    "http://localhost:5173",
],
```

Save the file and push:

```
► Run in: VS Code Terminal (project root)
git add .
git commit -m "Add Render frontend URL to CORS"
git push origin main
```

Render will auto-redeploy the backend within 2-3 minutes. Once redeployed, visit your frontend URL in the browser.

Step 5: End-to-End Verification

After all three services are deployed, run these checks to confirm everything is working.

5.1 Backend Health Check

```
► Run in: Mac Terminal (any location)
curl https://your-backend.onrender.com/health
```

Expected: `{"status": "healthy"}`

5.2 Database and API Check

```
► Run in: Mac Terminal (any location)
curl https://your-backend.onrender.com/api/v1/counties | head -c 300
```

Expected: JSON with total: 105 and county names. If total is 0, go back and re-run Step 2.

5.3 Frontend Visual Check

Visit your frontend URL in a browser. Open it in a fresh Incognito window (Cmd+Shift+N on Mac) to avoid cached data. Verify:

- The Kansas map renders with colored county shapes
- County colors show variation across metrics when switching between them
- The Compare tab dropdown is populated with county names
- The Analysis tab loads charts and statistics
- The Data Request tab submits successfully and sends a confirmation email

Known Issues and Limitations

Free Tier Spin-Down

Free tier web services spin down after 15 minutes of inactivity. The first request after spin-down can take 30-60 seconds while the service restarts. This is not configurable on the free tier. Upgrade to a paid Starter instance (\$7/month) to keep the service always running.

Dataset 3 Encoding Issue

Dataset 3 (DS/AI Course Inventory) uses latin1 encoding instead of UTF-8. The `_read_and_filter` function in `backend/app/api/data_request.py` must handle this. Open the file in VS Code and ensure it reads:

```
def _read_and_filter(csv_path, counties):  
    encoding = "latin1" if "dataset3" in csv_path.name else "utf-8"  
    df = pd.read_csv(csv_path, encoding=encoding)  
    ...
```

If this is not in place, requesting Dataset 3 in the Data Request tab will return a 500 error. After making this fix, push the change:

► Run in: VS Code Terminal (project root)

```
git add .  
git commit -m "Fix dataset3 latin1 encoding in data request"  
git push origin main
```

Excel Export Requires openpyxl

Excel (.xlsx) export in the Data Request tab requires the openpyxl library. Without it, any Excel export attempt returns a 500 Internal Server Error. Ensure openpyxl is present in `backend/requirements.txt`. If it is missing, add it locally and push:

```
cd backend && source venv/bin/activate  
pip install openpyxl && pip freeze > requirements.txt  
cd .. && git add backend/requirements.txt && git commit -m "Add openpyxl  
for Excel export" && git push origin main
```

Upgrading the PostgreSQL Database Plan

The free Render PostgreSQL database expires after 90 days and is permanently deleted with no grace period. Before it expires, back up your data and upgrade to a paid plan. The Basic tier (~\$7/month) is sufficient for the Kansas Atlas given its small database size.

Warning: Always back up your data before upgrading. If the free database has already expired and been deleted, you will need to reload from the CSV files using `load_data.py` as documented in Step 2.

Step 1: Back Up the Database

Render's PostgreSQL server runs version 18. Your local `pg_dump` must match this version or the backup will fail with a server version mismatch error. Install the correct version first:

```
► Run in: Mac Terminal (any location)
brew install postgresql@18
```

Then run the backup using the version-matched `pg_dump`, replacing `YOUR_EXTERNAL_DATABASE_URL` with the External Database URL from the Render dashboard (atlas-db → External Database URL):

```
/opt/homebrew/opt/postgresql@18/bin/pg_dump YOUR_EXTERNAL_DATABASE_URL >
atlas_backup.sql
```

Verify the backup file was created and is non-empty:

```
ls -lh atlas_backup.sql
```

Expected output: a file of several hundred KB (e.g. 363K). If the file is 0 bytes or missing, the backup failed — do not proceed with the upgrade until the backup succeeds.

Step 2: Upgrade the Plan in Render Dashboard

Once the backup is confirmed, upgrade the database in the Render dashboard. The upgrade happens in place with no data loss and no redeployment needed.

1. Go to dashboard.render.com → click atlas-db.
2. Click Change Plan (or Upgrade).
3. Select the Basic paid tier (~\$7/month) — sufficient for the Kansas Atlas.
4. Add a payment method if prompted → confirm the upgrade.

Note: Render introduced flexible plans in October 2024 where compute and storage are billed separately. If your database was created before this change it is on a legacy instance type. You can optionally migrate to a flexible plan from the Render dashboard, but this cannot be undone.

Redeployment and Future Updates

Once configured, updates to the application are handled automatically. No manual steps are needed on the Render dashboard for code changes:

- Code changes: Push to GitHub from VS Code Terminal (project root) — Render auto-deploys both frontend and backend
- Environment variable changes: Made in the Render dashboard — triggers an automatic redeploy
- Manual redeploy: From Render dashboard, click your service > Manual Deploy > Deploy latest commit

Re-loading the Database After a Schema Change

If you make changes to the database schema and need to reload all data:

18. Re-run create_tables.sql against the External Database URL to drop and recreate all tables:

► Run in: VS Code Terminal (project root)

```
psql YOUR_EXTERNAL_DATABASE_URL -f data/schemas/create_tables.sql
```

19. Re-run the data loader:

► Run in: VS Code Terminal (project root)

```
DATABASE_URL=YOUR_EXTERNAL_DATABASE_URL python backend/scripts/load_data.py
```

20. Verify the count:

► Run in: VS Code Terminal (project root)

```
psql YOUR_EXTERNAL_DATABASE_URL -c "SELECT count(*) FROM atlas;"
```

URL Reference

Replace the placeholders with your actual Render service names after deployment:

Key	Value
Frontend (public)	https://atlas-frontend-xxxx.onrender.com
Backend (public)	https://your-backend-name.onrender.com
Backend Health Check	https://your-backend-name.onrender.com/health
Backend API Docs (Swagger)	https://your-backend-name.onrender.com/docs
Counties Endpoint	https://your-backend-name.onrender.com/api/v1/counties
Heat Map Endpoint	https://your-backend-name.onrender.com/api/v1/visualizations/heat-map